

Python 工程化、Git 与命令行

Python engineering, Git & command line

本周导读 / Week overview

建立可维护 Python 项目的基础：环境隔离、文件与配置、错误处理、命令行界面和 Git 协作。

本周目标：把主题从“知道名词”推进到“能解释、能实现、能验证”。请先通读，再按章节完成实验

。

WEEK 01 · STUDY SECTION 02

知识地图 / Knowledge map

本周核心知识点：虚拟环境、pip、requirements.txt 与可复现安装；项目目录、模块导入、类型注解和函数边界；异常类型、错误信息、文件读写、JSON/CSV 和 UTF-8；PowerShell/Linux 的路径、进程、管道和退出码；git status/diff/add/commit/log、分支、合并、.gitignore

学习顺序：先建立最小概念模型，再阅读官方文档，随后写一个最小例子，最后把它接入本周项目。

学习记录模板：概念 | 我的解释 | 最小例子 | 常见错误 | 参考链接 | 是否验证

概念讲解 1 / Core concepts

虚拟环境、pip、requirements.txt 与可复现安装

解释：它描述了本周系统中的基本对象、边界和输入输出。学习时要同时记录一个正常例子和一个失败例子。

虚拟环境解决的是“同一台电脑上不同项目需要不同依赖版本”的问题。激活环境后，python 与 pip 都应指向环境目录；requirements.txt 记录的是项目依赖，不是 Python

本身。一个可复现流程是：创建环境、激活、安装、冻结版本、运行、退出。若 pip 安装成功但运行时找不到包，先检查解释器路径，而不是重复安装。

具体判断：当输入改变、执行顺序改变或依赖不可用时，虚拟环境、pip、requirements.txt 与可复现安装的结果会如何变化？请至少写出两个不同输入，并说明为什么结果不同。

小例子：为“成功、空输入、非法输入、重复操作、外部失败”各写一条记录。逐条记录预期行为、实际行为和证据；这比只展示一个成功截图更能证明理解。

检查答案：如果你只能说“框架会自动处理”，说明还没有掌握。继续追问数据在哪里、哪一层负责验证、错误如何传播，以及怎样通过测试观察到它。

概念讲解 2 / Core concepts

项目目录、模块导入、类型注解和函数边界

解释：把这个概念和项目中的一个函数、接口或数据结构对应起来。不要只抄 API；要说明为什么在这里需要它。

Git 的对象模型可以先理解为：工作区是你正在编辑的文件，暂存区是你准备提交的快照，提交是带作者、时间和父提交的不可变记录。一个好提交只完成一个可解释变化。分支只是移动到提交的指针；合并会把两个历史连接起来。

具体判断：当输入改变、执行顺序改变或依赖不可用时，项目目录、模块导入、类型注解和函数边界的结果会如何变化？请至少写出两个不同输入，并说明为什么结果不同。

小例子：为“成功、空输入、非法输入、重复操作、外部失败”各写一条记录。逐条记录预期行为、实际行为和证据；这比只展示一个成功截图更能证明理解。

检查答案：如果你只能说“框架会自动处理”，说明还没有掌握。继续追问数据在哪里、哪一层负责验证、错误如何传播，以及怎样通过测试观察到它。

概念讲解 3 / Core concepts

异常类型、错误信息、文件读写、JSON/CSV 和 UTF-8

解释：工程系统必须面对错误和不确定输入。设计时先问“错误在哪里被发现”，再问“用户看到什么响应”。

JSON 适合保存结构化的小数据，但写文件时如果进程中断，可能留下半个文件。可以先写临时文件，成功后替换原文件，并捕获 `JSONDecodeError`。Todo

的核心不在命令行，而在数据不变量：每条任务有唯一 id、标题、完成状态和创建时间。

具体判断：当输入改变、执行顺序改变或依赖不可用时，异常类型、错误信息、文件读写、JSON/CSV 和 UTF-8 的结果会如何变化？请至少写出两个不同输入，并说明为什么结果不同。

小例子：为“成功、空输入、非法输入、重复操作、外部失败”各写一条记录。逐条记录预期行为、实际行为和证据；这比只展示一个成功截图更能证明理解。

检查答案：如果你只能说“框架会自动处理”，说明还没有掌握。继续追问数据在哪里、哪一层负责验证、错误如何传播，以及怎样通过测试观察到它。

关键问题 / Questions

问题 1：Python 工程化、Git 与命令行最容易被初学者混淆的地方是什么？

问题 2：如果输入为空、类型不对、服务不可用或数据重复，系统应如何表现？

问题 3：怎样证明自己的实现确实有效，而不是只在一个示例上运行？

问题拆解：面对“Python 工程化、Git 与命令行”的题目，先画出对象和边界，再列出不变量。哪些值必须始终成立？哪些步骤可以重试？哪些错误必须立即停止？

面试式追问：为什么这个方案比最简单的硬编码更可靠？时间/空间成本是什么？如果输入规模扩大十倍，哪一部分先成为瓶颈？

学习要求：不要只看答案。先在纸上写出你的判断，再运行最小实验验证；若结果和预期不同，保留这个反例并解释原因。

请把答案写在自己的学习笔记中，再对照本页检查，不要只阅读答案。

问题解答 / Answers

解答：先给概念下可操作的定义，再用输入、处理、输出三段描述。边界情况必须成为测试，而不是等到演示时临时处理。

解答：把失败划分为用户输入错误、外部依赖错误和程序缺陷；分别返回可理解提示、可重试响应或记录完整诊断。

解答：使用独立测试、固定输入、可重复命令、日志和验收清单，避免只凭肉眼看一次输出。

详细解答方法：先定义术语，再给出执行流程，最后说明边界情况。解答中应出现至少一个输入、一个中间状态和一个输出，而不是只有结论。

如果答案涉及性能，请给出操作数量的增长趋势；如果涉及可靠性，请给出失败检测、恢复和再次运行的行为；如果涉及模型，请区分检索错误和生成错误。

自测：遮住本页后，用两分钟向别人讲解；对方应能根据你的讲解写出一个最小测试。做不到时回到概念页补充。

请把答案写在自己的学习笔记中，再对照本页检查，不要只阅读答案。

具体示例 / Worked example

场景：开发一个 JSON 文件版命令行 Todo

应用：新增、列表、搜索、完成、删除；处理文件不存在、JSON 损坏、重复 ID 和非法输入。每完成一个垂直功能就提交一次。

步骤 1：写出最小输入和预期输出；步骤 2：实现一条成功路径；步骤 3：加入一个失败输入；步骤 4：写测试或记录复现命令；步骤 5：提交代码并记录原因。

示例分析：如果结果不符合预期，先定位是输入、状态、依赖还是断言问题，再做最小修复，不要同时改动多个模块。

逐步示例：围绕“开发一个 JSON 文件版命令行 Todo

应用：新增、列表、搜索、完成、删除；处理文件不存在、JSON 损坏、重复 ID 和非法输入。每完成一个垂直功能就提交一次。”，先构造最小输入。观察每一步的变量、数据结构和副作用；在关键节点打印或记录可诊断信息。

变体一：把输入改为空值或极端值；变体二：让外部依赖返回错误；变体三：重复执行同一操作。比较三种情况下系统是否安全、可解释、可恢复。

示例结论必须包含限制：这个示例证明了什么，没有证明什么，下一步需要怎样的测试或真实数据。

代码/流程示例 / Implementation example

伪代码：读取输入 → 验证输入 → 调用核心逻辑 → 保存或返回结果 → 记录可诊断日志。

每一步都应有清晰的职责和可观察结果。核心逻辑尽量不直接依赖终端、网络或真实模型，这样可以用小测试替换外部依赖。

请把伪代码翻译成你本周使用的语言和框架，并在 README 中放一个最小调用示例。

实现时要区分“展示代码”和“生产代码”。展示代码帮助理解单个概念；生产代码还需要输入验证、错误处理、日志、测试、配置和可重复启动。

建议先写接口契约：输入字段、输出字段、状态码/错误类型、持久化规则和超时。契约稳定后再选择具体库，避免让框架行为替代自己的设计。

代码审查问题：这段逻辑能否单独测试？换成假的数据库或模型客户端是否仍能测试？异常是否会泄露密钥或用户内容？

动手实验 / Guided lab

开发一个 JSON 文件版命令行 Todo 应用：新增、列表、搜索、完成、删除；处理文件不存在、JSON 损坏、重复 ID 和非法输入。每完成一个垂直功能就提交一次。

实验要求：先建立最小版本，再分 3

次增加验证、错误处理和测试。每次运行都记录命令、输入、输出和耗时；每次修改都做一次 Git commit。

实验记录模板：日期、环境、命令、输入、预期、实际、日志、结论、下一步。只要有一个字段缺失，未来就很难解释为什么某次结果不同。

完成最小版本后，故意制造一个错误并观察系统。可靠性学习的关键不是永远不出错，而是出错时能快速发现、清晰说明并安全恢复。

完成实验后将结果写进 README 或学习日志，附上真实命令和截图，不要用“应该可以”替代证据。

练习题 / Exercises

练习 1：用自己的话解释“虚拟环境、pip、requirements.txt 与可复现安装”并写一个最小例子。

练习 2：为项目设计 3 个正常输入、2 个边界输入和 2 个失败输入。

练习 3：画出一请求或程序运行的流程图，标明数据在哪里产生、改变和保存。

练习 4：指出一个你会选择的替代方案，并比较它的成本、风险和适用范围。

练习必须先独立完成。写出你的解法、复杂度和失败案例后，再对照参考答案。参考答案用于发现遗漏，不是替代思考。

评分标准：概念准确 30%，示例可运行 25%，边界测试 20%，解释技术取舍 15%，记录和复现 10%。

如果不会，记录“卡点类型”：术语不懂、流程不清、代码错误、环境问题或无法设计测试；不同卡点需要不同补救。

请把答案写在自己的学习笔记中，再对照本页检查，不要只阅读答案。

WEEK 01 · STUDY SECTION 12

练习参考答案 / Model answers

答案思路：概念解释必须包含用途、输入输出和限制；最小例子只保留证明概念所需的代码。

答案思路：正常输入验证主要功能，边界输入验证空值/极值/重复，失败输入验证错误响应和恢复方式。

答案思路：流程图至少包含入口、验证、核心处理、持久化/返回和错误分支。

答案思路：替代方案不能只写名称，要说明在当前项目规模下为什么选择或暂不选择。

参考答案不是唯一实现。判断答案质量时，检查它是否满足题目约束、是否处理边界、是否能被测试，以及是否解释了为什么这样选择。

把参考答案改写成自己的版本：替换变量、数据、错误路径和测试；如果只能原样抄写，说明仍需回到概念和示例。

最后写一个“我仍然不确定的地方”，下一周用实验、文档或面试题把它变成确定结论。

请把答案写在自己的学习笔记中，再对照本页检查，不要只阅读答案。

常见错误 / Failure modes

把“能运行一次”当成完成；忽略环境、版本、数据和随机性。

把异常吞掉或返回模糊的“出错了”，让用户和开发者都无法处理。

把业务逻辑、输入输出、数据库和外部 API 写在同一个超长函数里。

为了通过一个样例硬编码结果，导致换输入就失败。

把未测试、未部署或计划中的内容写成已完成成果。

错误分类：概念误解、边界遗漏、状态管理错误、依赖配置错误、并发/性能问题、测试假设错误。先分类再修复，避免盲目改代码。

修复记录应包含复现步骤、根因、最小修复、回归测试和仍然存在的风险。没有回归测试的修复很容易在下一次改动中再次失败。

可交付成果 / Deliverables

README、依赖清单、示例数据、运行说明、至少 5 次有意义提交；程序可以从新目录安装后运行。

交付检查：源码可运行；README

可复现；测试或实验记录可核对；截图/日志不泄露密钥；所有性能和准确率数字都有测量条件。

交付物必须能被别人验证：源码、启动命令、依赖版本、测试命令、示例输入输出和已知限制缺一不可。只有截图而没有复现步骤，不能算完整交付。

如果某项尚未完成，明确写 Planned /

计划中，并列出发完成它所需的下一步。诚实的限制说明比虚构指标更能体现工程判断。

验收与复盘 / Check & reflection

逐项验收：创建 venv 并激活，确认 python 与 pip 指向虚拟环境。；用 argparse

或标准库实现清晰的子命令和 --help。；为 JSON

读写封装函数，写入时使用临时文件或明确处理失败。；写 5

个手工测试场景并记录预期与实际结果。；检查 git diff，确认没有提交虚拟环境、密钥和生成文件。

完成后写下三个最重要概念、一个失败根因、一个保留的工程决策和一个下周问题。状态只有在证据齐全后才可勾选。

复盘时不要只写“学到了很多”。写出一个具体前后对比：之前会犯什么错误，现在如何通过定义、示例、测试或日志避免它。

把仍不熟练的点安排到下一周的第一项任务，并在下次复盘时检查是否真的解决。这样 PDF 才会变成可执行的学习系统，而不是一次性阅读材料。